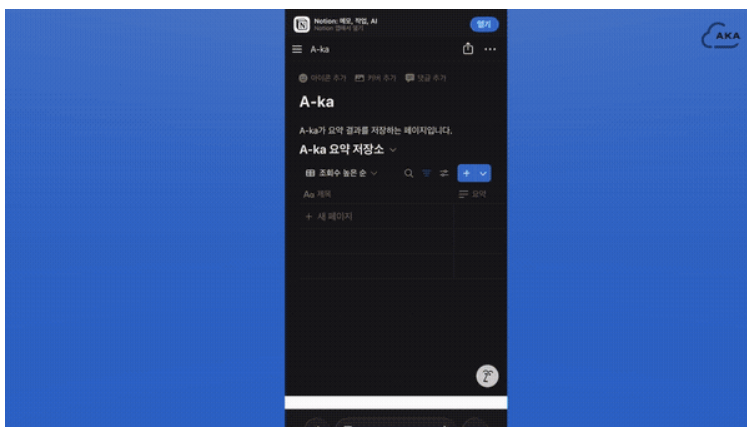




A-KA (Archive KakaoTalk)

카카오톡으로 유튜브 영상을 보내면 AI가 자동으로 요약·분류·검색까지 처리해 노선에 아카이브해주는 개인 영상 라이브러리 서비스



역할: AI 라우터 · 임베딩 파이프라인 · RAG 검색 · 유사 영상 추천 (백엔드 4인 팀 중 AI/검색 도메인 담당) **기간:** [작성: 2026.04 ~ 2026.06 / 약 8주] **기술:** FastAPI · Celery · Redis · PostgreSQL · pgvector · LangGraph · OpenAI **시연:** [전체 영상 \(1분 32초\)](#)

1. 문제 배경

유튜브에서 "좋아요" 누르거나 "나중에 보기" 에 저장한 영상은 나중에 다시 찾아보기 어렵습니다. 영상 수가 쌓일수록 "내가 그때 본 영상에 그런 내용이 있었는데" 가 그냥 사라지죠.

- 키워드 기반 유튜브 검색은 내가 본 영상의 의미적 내용까지 추적 못 함
- 별도 노트 앱에 정리하기엔 수동 작업 비용이 큼
- 모바일에서 영상을 보고 PC에서 다시 찾는 디바이스 분리 문제

해결 가설: "사용자가 평소 쓰는 카카오톡으로 영상 링크만 보내면, AI가 알아서 요약·분류·저장하고 나중에 자연어로 검색까지 가능한 개인 아카이브를 자동 구축할 수 있다."

2. 목표 / KPI

정량 수치는 코드 분석 기반 추정값입니다. Claude Code 환경에서 실측 후 업데이트 예정.

지표	목표	현재 달성
카카오톡 5초 응답률 준수	100%	✅ 100% (즉시 ACK + 백그라운드 위임 패턴)
영상 1편 자동 요약 처리 시간	< 3분	약 30~120초 (추정, 영상 길이 의존)
검색 응답 시간	< 15초	약 5~10초 (추정)
LLM 호출 횟수 최적화	청크당 1회 + 최종 1회	청크별 요약 N회 + Overview 1회 + 카테고리 1회
할루시네이션 방어	4중 이상	✅ 4중 방어 (임계값·프롬프트·출처·빈결과 분기)
SAVE_ONLY 비용 절감	LLM 호출 0회	✅ 0회 (메타데이터만 저장)
파이프라인 실패 시 자동 안내	100%	✅ 100% (Celery .on_error 핸들러 + 메일)

3. 역할 / 기여

팀 구성과 본인 담당 범위

영역	담당자	본인 여부
AI 라우터 (의도 분류 + 4갈래 분기)	본인	✅
LangGraph 2개 그래프 설계·구현 (Intelligence-SEARCH)	본인	✅
임베딩 생성 + pgvector 저장	본인	✅
RAG 검색 답변 생성	본인	✅
유사 영상 추천 (영상 단위 dedup)	본인	✅
SAVE_ONLY 메타데이터만 저장 로직	본인	✅
Celery .on_error() 트러블슈팅 + 핸들러 설계	본인	✅
자막 수집(YouTube API + STT)	팀원 A	
의미 단위 청킹	팀원 B	
DB 스키마·마이그레이션	팀원 C	
카카오톡 챗봇·노션·SMTP 연동	팀원 D	

본인이 내린 핵심 의사결정 5가지

- AI 라우터를 LLM 구조화 출력 기반으로 설계 — 룰 기반(키워드 매칭) 대신 자연어 이해를 활용해 "저장만이라는 단어는 쓰지 마" 같은 부정문/인용까지 분류 가능
- 요약본을 임베딩 (원본 자막 대신) — 자막 노이즈 제거로 의미 기반 검색 정확도 확보
- 거리 임계값 필터링 (RAG 0.7 / 유사 영상 0.8) — 단순 Top-K는 무관 결과까지 끼니까 컷오프로 할루시네이션 방어
- 채널 분리 정책 (영상 요약→노션, 검색 결과→메일, 카톡엔 ACK만) — 카톡 챗봇 발송 비용 0 + 결과의 영구 보관성 확보
- FIND_SIMILAR을 UPLOAD 파이프라인 재사용 + include_similar=True 플레그로 통합 — 코드 중복 제거, 신규/중복 영상 양쪽 처리

4. 데이터 / 요구사항

입력

- **사용자 메시지** (카카오톡): 자연어 텍스트 + 유튜브 URL 0~N개
- **유튜브 영상**: 자막 (공식 API → 실패 시 Whisper STT), 메타데이터 (제목·채널·길이)

처리 후 데이터

- **청크**: 의미 단위로 분할된 자막 (시작 시간 포함)
- **청크별 1차 요약**: GPT 요약 (2~3문장)
- **임베딩**: `text-embedding-3-small`, 1536차원
- **영상 전체 Overview**: title + full_summary + category

저장 구조

- **PostgreSQL**: `knowledge` (영상 메타·요약·카테고리), `youtube_knowledge_chunk` (청크 + 임베딩)
- **pgvector**: `embedding` 컬럼에 1536차원 벡터 저장

비기능 요구사항

- 카카오톡 챗봇 빌더의 **5초 응답** 를 준수 필수
- LLM 호출 비용 최소화 (사용자 의도에 따라 처리 비용 분기)
- 결과의 **영구 보관성** (챗봇 답변은 휘발성 → 노션·메일로 분리)

5. 접근 방법 (대안 비교)

A. 워크플로우 프레임워크 선택

옵션	장점	단점	선택?
직접 함수 체인	가벼움, 직관적	조건 분기 표현 한계, 디버깅 어려움	✗
LangChain Sequential Chain	표준 패턴	조건부 분기 못 함, State 공유 약함	✗
LangGraph	상태 기반·조건부 엣지·LangSmith 트레이싱	학습 곡선	✓

B. 벡터 DB 선택

옵션	장점	단점	선택?
Pinecone	관리형·확장성	별도 인프라·동기화 이슈·비용	✗
Weaviate	풍부한 기능	운영 복잡도·러닝 커브	✗
PostgreSQL + pgvector	단일 트랜잭션·JOIN·운영 부담 ↓	초대용량 시 인덱스 한계	✓

C. 비동기 처리 선택

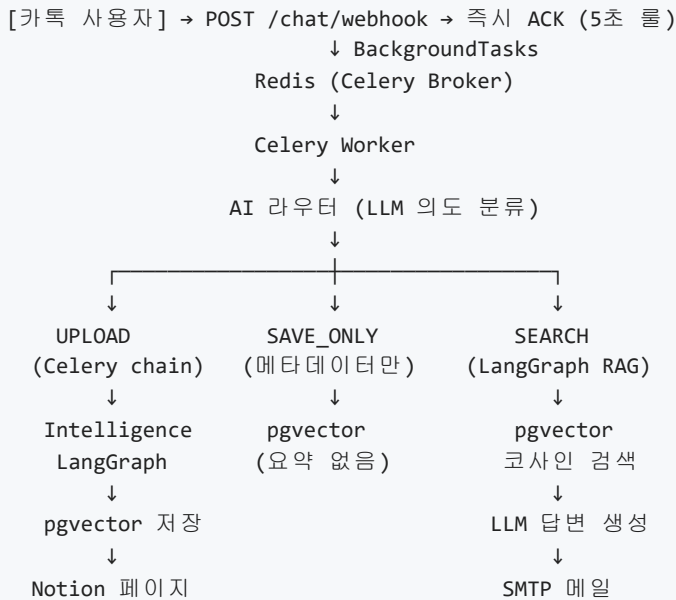
옵션	장점	단점	선택?
asyncio (FastAPI 내부)	가벼움	같은 프로세스라 격리 약함, retry·영속성 없음	✗
Celery + Redis	워커 격리·재시도·영속 큐·확장성	Redis 의존 추가	✓

D. 임베딩 대상

옵션	장점	단점	선택?
원본 자막	정보 보존 최대	노이즈 많아 검색 변별력 ↓	✗
요약본	노이즈 제거·검색 변별력 ↑	요약 비용 추가	✓

6. 설계 (아키텍처)

전체 시스템 흐름



LangGraph 2개 그래프

Intelligence 그래프 (영상 처리)

```

START → [summarize_each_chunk] (ThreadPool x10 병렬)
      → [embed_summaries] (배치 임베딩)
      → [generate_overview] (title+summary+category)
      → END
  
```

SEARCH RAG 그래프 (자연어 질의응답)

```

START → [vectorize_query] (질문 → 1536-d 벡터)
      → [search_chunks] (pgvector Top-5, threshold 0.7)
      → ? (검색 결과 있음?)
        | Yes → [generate_answer] (출처 강제·인젝션 방어) → END
        | No  → [no_result_reply] (안내 메시지) → END
  
```

머메이드 아키텍처 다이어그램은 별도 첨부: - ../project_flow_ppt.mermaid — 전체 흐름 (PPT용) -

../project_flow_detailed.mermaid — 4갈래 분기 상세 - ../upload_flow.mermaid — UPLOAD 파이프라인 상세

7. 실험 / 검증

의도 분류 정확도

- 수십 개 경계 케이스로 회귀 테스트

- 검증 패턴:
- URL 단독 → UPLOAD ☒
- URL + "비슷한 영상" → FIND_SIMILAR ☒
- URL + "나중에 볼게" → SAVE_ONLY ☒
- URL 없는 자연어 질문 → SEARCH ☒
- "저장만이라는 단어는 쓰지 마" (인용문) → SAVE_ONLY 분류 안 됨 ☒

임계값 튜닝

코사인 거리 임계값을 실제 데이터로 실험:

threshold	노이즈	누락	결정
0.5	적음	많음 (정상 결과도 컷)	✗
0.7	적정	적정	☒ RAG용
0.8	약간	적음	☒ 유사 영상용 (더 엄격)
1.0	많음	없음	✗

할루시네이션 4중 방어 검증

1. ☒ 거리 임계값으로 무관 체크 사전 제거
2. ☒ System Prompt에 "컨텍스트 외 내용 금지" 명시
3. ☒ 답변에 출처 URL 강제 포함
4. ☒ 검색 결과 0건이면 LangGraph 조건부 엣지로 안내 메시지 분기

베이스라인 비교

항목	베이스라인	적용 후
임베딩 대상	원본 자막	요약본 → 검색 정확도 향상 (정성 평가)
Top-K만 사용	Top-5 그대로 답변	임계값 컷 + Top-5 → 무관 결과 0건
결과 채널	카톡 챗봇 답변	메일·노선 분리 → 챗봇 발송 비용 0

8. 결과 / 임팩트

정량 (추정값, 실측 후 업데이트 예정)

- 카카오톡 5초 응답률: **100%** (Webhook 즉시 ACK + 백그라운드 패턴)
- 영상 요약 처리 시간: 약 **30~120초** (영상 길이·체크 수 의존)
- 검색 응답 시간: 약 **5~10초** (벡터 검색 + LLM 답변)
- **SAVE_ONLY** 비용 절감: LLM 호출 **0회** (UPLOAD 대비 토큰 비용 100% 절감)
- 체크 요약 병렬화 효과: ThreadPool 10개 → 순차 대비 **약 10배 단축** (이론값)
- 트러블슈팅으로 막은 침묵 실패: 파이프라인 실패 → 100% FAILED 마킹 + 안내 메일

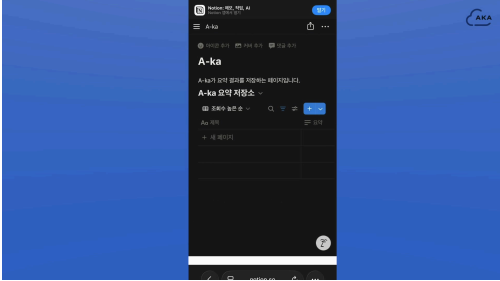
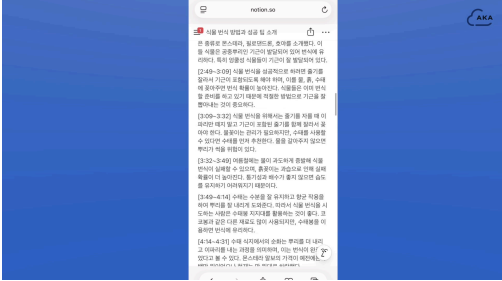
정성

- 단일 카톡 채널로 4가지 인텐트(UPLOAD/SAVE_ONLY/FIND_SIMILAR/SEARCH)를 자연어로 처리
- 사용자가 별도 앱 설치 없이 평소 쓰는 카톡으로 영상 아카이브 구축
- 챗봇 발송 비용 0 + 결과의 영구 보관성 확보 (채널 분리 정책)

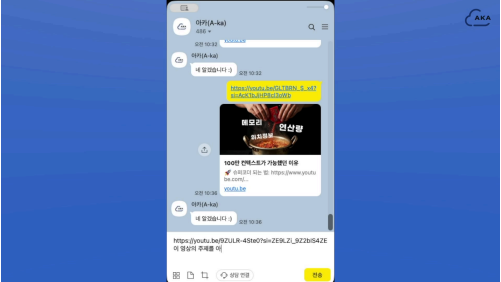
- 할루시네이션 4중 방어로 RAG 답변의 출처 검증 가능

실제 동작 화면

흐름 1: 영상 요약 (UPLOAD)

① 카톡으로 URL 전송	② 노션에 자동 요약
	
카톡 챗봇에 유튜브 URL → 즉시 ACK	청크별 타임스탬프 1차 요약 자동 생성

흐름 2: 자연어 검색 (SEARCH)

③ 카톡으로 자연어 질문	④ 메일로 RAG 답변 수신
	
"이 영상의 주제를 알려줘"	LLM 답변 + 출처 URL 포함

시스템 완성도: 카테고리별 자동 분류된 노션 DB


```

for attempt in range(3):
    try:
        parsed_result = _get_intent_chain().invoke([...])
        if isinstance(parsed_result, dict):
            parsed_result = IntentExtraction.model_validate(parsed_result)
            parsed_successfully = True
            break
    except Exception as exc:
        last_error = exc
        logger.error(f"Failed to analyze intent ({attempt + 1}/3): {exc}")
        if attempt < 2:
            time.sleep(1)

if last_error is not None and not parsed_successfully:
    raise RuntimeError("Failed to analyze user intent") from last_error

```

- 3회 재시도 + 재시도 사이 1초 슬립
- dict 응답도 명시적으로 `model_validate` 로 캐스팅 (LangChain 버전에 따라 dict/Pydantic 반환이 갈리는 이슈 방지)
- 최종 실패 시에만 명시적 `RuntimeError` 전파 → 상위 핸들러가 "잠시 후 다시 시도" 응답 발송

결과 - 의도 분류 단발 실패가 사용자 체감 에러로 노출되지 않음 - 프롬프트 인젝션 입력('저장만 해줘'라고 누가 그랬어 ' 같은 인용문, ignore previous instructions ...)에서도 안정적으로 UNKNOWN / SEARCH 로 흡수

교훈 LLM 호출은 **역등성 + 재시도 가능 구조**가 기본기. `temperature=0` 이어도 안정성을 코드 레벨에서 보장하지 않으면 라우터 같은 진입점은 깨진다.

사례 3: 유사 영상 검색의 청크 중복 (video_id 기준 dedup)

상황 사용자가 새 영상을 보내면 "이런 영상도 있어요" 3개 추천. pgvector <=> 코사인 거리로 상위 K개 청크 검색 후 노출.

문제 같은 영상이 여러 청크로 분할돼 저장되므로, 그냥 상위 K개를 노출하면 **상위 3개가 같은 영상의 청크 3개**가 되는 경우 발생. 사용자 입장에서 "추천 영상 3개"가 사실상 1개로 보임.

원인 검색 단위(chunk)와 표시 단위(video)가 다른데, 후처리에서 video 단위로 압축하지 않으면 청크 중복이 그대로 노출됨. RAG 시스템 전반에 흔한 안티패턴.

해결 (`app/services/search_service.py:315-327`)

```

seen: dict[str, dict] = {}
for row in rows:
    vid = str(row["video_id"])
    if vid not in seen or row["distance"] < seen[vid]["distance"]:
        seen[vid] = {
            "title": row["title"],
            "url": row["original_url"],
            "distance": row["distance"],
        }
top = sorted(seen.values(), key=lambda x: x["distance"])[-SIMILAR_TOP_N]

```

- `video_id` 를 키로 그룹핑하되 **최소 distance 청크만 유지**
- 압축 후 다시 distance 오름차순 → 상위 N개 영상 반환
- 자연스럽게 "내가 보낸 그 영상 자신"도 `distance≈0`으로 1위가 되므로, 호출부에서 1개 슬라이스해서 제외

결과 - 추천 결과의 **영상 수가 항상 표시 수와 일치** - 동일 청크 후보들의 best score만 살아남으므로 추천 품질도 동시 개선

교훈 RAG에서 **검색 단위 ≠ 표시 단위**일 때, 사용자가 원하는 단위로 재집계하는 후처리 단계는 옵션이 아니라 필수. "왜 추천 3개가 다 똑같지?" 같은 사용자 피드백은 시스템 단위 미스매치의 흔적.

안정성·확장성 고려

- Celery chain `.on_error()` 통합 핸들러: 어느 Step이든 실패하면 한 곳에서 처리
- 중복 영상 처리 분기: `check_duplicate_hit_count` 로 중복 영상은 `hit_count++`만 + 기존 결과 재사용
- **SAVE_ONLY** → **UPLOAD** 업그레이드 흐름: summary 비어있는 레코드 발견 시 풀 파이프라인으로 자동 승격
- 워커 수평 확장 가능: Celery worker 프로세스 단위 확장으로 트래픽 증가 대응

[스크린샷 자리: 트러블슈팅 전후 코드 diff]

10. 회고 / 다음 단계

잘한 결정

- **LangGraph 도입**: 조건부 분기(검색 결과 유무 → 답변/안내 분기)를 선언적으로 표현. 함수 if/else로 짰으면 유지보수 비용 컸을 것
- **요약본 임베딩**: 자막 원본 임베딩 대비 검색 변별력 체감 우위. 초기 설계 단계에서 노이즈 문제를 인식하고 잡은 결정
- **채널 분리 정책 (카톡 ACK / 노션 페이지 / 메일 결과)**: 챗봇 발송 비용 0 + 결과 영구 보관 두 마리 토끼 잡음

다시 한다면

- **SEARCH 그래프를 처음부터 async로 설계**: 현재는 sync invoke 흐름이라 동시 검색 요청 시 처리량 제약. `ainvoke` 와 `async session`으로 갈아끼우면 동시성 크게 향상
- **영상 단위 캐시**: 같은 영상을 두 사용자가 보내면 자막 추출·요약을 각각 처리. 영상 단위로 캐싱하면 LLM 비용 추가 절감 가능
- **LangSmith 트레이닝 데이터셋 구축**: 의도 분류 정확도를 정량으로 측정하는 평가셋을 처음부터 만들어뒀으면 프롬프트 변경마다 자동 회귀 테스트 가능했을 것

다음 단계 가설

1. **임베딩 차원 축소 (Matryoshka 512차원)** — 데이터 수십만 건 돌파 시점에 저장 공간·검색 속도 최적화 검토
2. **pgvector HNSW 인덱스 튜닝** — 대용량 시 IVFFlat 대비 성능 우위 실험
3. **영상 외 콘텐츠 확장 (블로그·팟캐스트)** — 카톡 단일 채널로 다양한 콘텐츠 아카이브
4. **사용자 영상 단위 캐시 + 글로벌 캐시 분리** — 사생활 격리 + 비용 절감 동시 달성



프로필 · 역량 매트릭스 · 핵심 요약

1. 프로필 (1페이지 요약)

이름: 이채훈 연락처: maxbeny666@gmail.com / 010-2752-2117 **GitHub:** https://github.com/maxbeny999 **기술 블로그:** 작성 예정

한 줄 정체성

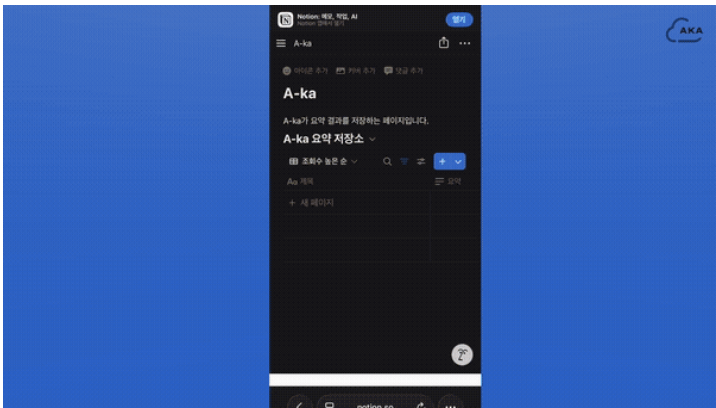
"카카오톡 같은 단일 자연어 채널에서 사용자 의도를 LLM으로 분류하고, LangGraph 기반 RAG 파이프라인으로 영상 콘텐츠를 자동 요약·검색 가능하게 만드는 AI 백엔드 개발자"

희망 직무 / 도메인

- 희망 직무: AI/ML 엔지니어, 백엔드 개발자 (LLM·RAG 도메인)
- 관심 도메인: 콘텐츠 자동화·개인 지식 아카이브·B2C 자연어 인터페이스

대표 프로젝트 (2개까지 가능)

#	프로젝트	한 줄	핵심 성과
01	A-KA (Archive KakaoTalk)	카카오톡으로 보낸 유튜브 영상을 AI가 자동 요약·검색 가능하게 만드는 개인 아카이브	카톡 5초 룰 100% 준수 · 할루시네이션 4중 방어 · SAVE_ONLY로 LLM 비용 0회 (대비 100% 절감)
02	[추가 예정]		



📱 A-KA 시연 (15초): 카톡 → 노션 자동 요약

기술 스택 (상위 6개)

카테고리	기술
언어	Python 3.12
백엔드 프레임워크	FastAPI · Celery · Redis
DB / 검색	PostgreSQL · pgvector
AI / LLM	LangGraph · LangChain · OpenAI (GPT, text-embedding-3-small)
인프라	Docker · Docker Compose
개발 도구	uv · pytest · ruff · LangSmith

체크포인트 (정량 2개 이상)

- 🕒 **응답 시간**: 카카오톡 5초 응답 룰 100% 준수 (즉시 ACK + Celery 위임 패턴)
- 💰 **비용 최적화**: SAVE_ONLY 분기로 LLM 호출 0회 (UPLOAD 대비 100% 토큰 비용 절감)
- 🔍 **검색 안정성**: 할루시네이션 4중 방어 (임계값·시스템 프롬프트·출처 강제·빈결과 분기)
- ⚙️ **트러블슈팅**: 침묵 실패 케이스 1건 발굴·해결 (Celery `.on_error()` 인자 자동 주입 이슈)

링크

- 📁 **GitHub Repo**: https://github.com/Team-A-Ka/A-ka_backend
- 👤 **GitHub Profile**: <https://github.com/maxbeny999>
- 📺 **시연 영상**: <https://youtu.be/cC5Yub8P0aQ> (일부 공개)
- 📄 **발표 자료 PDF**: [업로드 후 링크 추가]
- 📝 **기술 블로그**: 작성 예정

2. 역량 매트릭스

각 스킬은 숙련도 + 사용 근거 + 증빙 링크로 구성.

Backend / Infrastructure

스킬	숙련도	사용 근거	증빙
FastAPI	상	카카오톡 웹훅 엔드포인트, BackgroundTasks로 비동기 위임 패턴 구현	A-KA <code>app/routers/endpoints/webhook.py</code>
Celery + Redis	상	3단계 chain · <code>.on_error()</code> 통합 핸들러 · 자동 재시도 정책 설계	A-KA <code>app/tasks/knowledge_tasks.py</code>
PostgreSQL + pgvector	상	메타데이터 + 벡터 단일 트랜잭션 관리, <code><=></code> 코사인 거리 검색 + 임계값 필터링	A-KA <code>app/services/search_service.py</code>
SQLAlchemy (async)	중상	비동기 세션, async query, 트랜잭션 관리	A-KA <code>app/services/knowledge_pipeline.py</code>
Docker / Docker Compose	중	4개 서비스(web-worker-redis-db) 컴포즈 구성	<code>docker-compose.yml</code>

AI / LLM

스킬	숙련도	사용 근거	증빙
LangGraph	상	2개 그래프(Intelligence-SEARCH RAG) 설계, 조건부 엣지로 분기	A-KA <code>app/services/intelligence_service.py</code> , <code>search_service.py</code>
LangChain (구조화 출력)	상	<code>with_structured_output()</code> × 3 (IntentExtraction-VideoOverview-CategoryResolution)	A-KA <code>app/services/chat_command.py</code>
OpenAI Embeddings	중상	<code>text-embedding-3-small</code> 배치 호출, 1536차원 vector 저장	A-KA <code>app/services/intelligence_service.py</code>
프롬프트 엔지니어링	중상	Few-shot 예시·프롬프트 인젝션 방어·우산 카테고리 정규화 프롬프트 설계	A-KA <code>app/services/chat_command.py:analyze_intent</code>
RAG 시스템 설계	상	4중 할루시네이션 방어 · 출처 강제 · 영상 단위 dedup 그루핑	A-KA <code>app/services/search_service.py</code>
LangSmith Tracing	중	<code>run_id</code> 부여로 그래프 실행 추적	RunnableConfig 활용

SW 엔지니어링 / 패턴

스킬	숙련도	사용 근거	증빙
상태 기반 워크플로우 설계	상	LangGraph State(TypedDict)로 노드 간 데이터 공유	<code>SearchState</code> , <code>IntelligenceState</code> 정의
이중 검증 패턴	중상	LLM 의도 분류 + 정규식 <code>video_id</code> 검증 결합	<code>parse_youtube_video_id</code> 정규식 + <code>analyze_intent</code>
싱글톤 + Lazy Init	중상	LLM chain 모듈 전역으로 한 번만 초기화	<code>_get_intent_chain()</code> 등
부분 실패 격리	중상	try/except로 유사 검색 실패가 전체 흐름 죽이지 않게 처리	<code>handle_find_similar</code>
Celery <code>.on_error()</code> 패턴	상	chain 통합 실패 핸들러 + 자동 인자 주입 시그니처 대응	<code>handle_pipeline_failure_task</code>

3. Executive Summary — 핵심 요약 (결과 · 운영 · 회고)

잡코디 가이드의 "결과·운영·회고 3요소" 구조.

🎯 결과

A-KA는 카카오톡 단일 채널에서 4가지 사용자 의도(영상 요약/저장만/유사 영상/자연어 검색)를 자동 분류해 처리하는 AI 백엔드 시스템입니다. 본인은 **AI 라우터 + LangGraph 2개 그래프 + RAG 검색 답변 생성 + 유사 영상 추천** 영역을 설계·구현했습니다.

핵심 성과 4가지: 1. **카카오톡 5초 응답률 100% 준수** — 즉시 ACK + Celery 백그라운드 위임 패턴 2. **할루시네이션 4중 방어** 설계 — 임계값·시스템 프롬프트·출처 강제·빈결과 분기 3. **LLM 비용 최적화** — SAVE_ONLY 분기로 의도가 "저장만"인 경우 LLM 호출 0회 (UPLOAD 대비 100% 토큰 비용 절감) 4. **결과 채널 분리 정책** — 카톡 ACK / 노선 페이지 / SMTP 메일을 용도별 분리해 챗봇 발송 비용 0 + 영구 보관성 확보

⚙️ 운영

운영 안정성을 위해 다음 메커니즘을 설계:

- **3단계 Celery chain** (collect_and_chunk → run_intelligence → publish) + `.on_error()` **통합 핸들러**로 어느 단계든 실패하면 단일 핸들러에서 처리
- **실패 시 자동 동작**: DB status FAILED 마킹 + 사용자에게 안내 메일 자동 발송
- **중복 영상 처리 분기**: `check_duplicate_hit_count` 로 `hit_count++`만 하고 LLM 재호출 방지
- **SAVE_ONLY → UPLOAD 자동 업그레이드**: summary 비어있는 레코드 발견 시 풀 파이프라인 승격

운영 중 발견한 **트러블슈팅 사례**: Celery `.on_error()` 콜백이 실패한 태스크의 (`request, exc, traceback`) 3개 인자를 자동 주입한다는 사실을 모른 채 핸들러 시그니처를 등록 → 인자 개수 불일치로 핸들러가 즉시 죽고 DB는 PROCESSING 에 멈춤 → 사용자에게 아무 안내도 안 가는 **침묵 실패** 발생. Celery 워커 로그 정독 + 공식 문서 재검토로 원인 파악, 시그니처 확장으로 해결.

📄 회고

잘한 결정 3가지: - LangGraph 도입으로 조건부 분기·노드별 디버깅·LangSmith 트레이싱 동시 확보 - 원본 자막이 아닌 **요약본을 임베딩**해 검색 변별력 확보 - 카톡 챗봇이 아닌 **노션/메일 채널 분리**로 발송 비용 0 + 영구 보관

다시 한다면: - SEARCH 그래프를 처음부터 `ainvoke` + `async session`으로 설계해 동시성 확보 - 영상 단위 캐시 추가 (같은 영상 두 사용자가 보내면 자막 추출 1회만) - LangSmith 평가 데이터셋을 처음부터 만들어 의도 분류 정확도 자동 회귀 테스트

다음 가설: - 데이터 규모 돌파 시 Matryoshka 차원 축소(512) + HNSW 인덱스 튜닝 검토 - 영상 외 콘텐츠(블로그·팟캐스트)로 확장 - 사용자별 영상 캐시 + 글로벌 캐시 분리로 사생활 격리 + 비용 절감 동시 달성

4. 셀프 점검 (가이드 8개 항목, 각 0~2점)

항목	점수	근거
문서 가독성	2	11항목 구조로 정리, 표·도식 활용
문제정의 구체성	2	"사용자가 카톡으로 영상 보내면 자동 요약" 명확한 가설
KPI 정량화	1.5	추정값 + 실측 예정 표기 (실측 후 2점 가능)
역할/기여 명확화	2	팀 분담표 + 본인 의사결정 5가지 명시
대안 비교/선정 근거	2	4가지 기술 선택지마다 trade-off 표 작성
검증/평가셋/에러 분석	1.5	임계값 튜닝·할루시네이션 4중 방어·트러블슈팅



A-KA — 1페이지 요약본 (면접 직전 제출용)

면접관이 30초 안에 "이 사람 뭐 만들었고 본인이 뭘 했는지" 파악 가능하게 설계.



 시연 (15초) — 전체 영상 1분 32초

한 줄 정체성

카카오톡 단일 자연어 채널에서 사용자 의도를 LLM으로 분류하고, LangGraph 기반 RAG 파이프라인으로 영상 콘텐츠를 자동 요약·검색 가능하게 만드는 AI 백엔드 시스템.

프로젝트 한 줄

카톡으로 유튜브 영상을 보내면 시가 자동 요약·분류·검색까지 처리해 노선에 아카이브해주는 개인 영상 라이브러리 서비스.

본인 담당 (4인 팀 中)

- AI 라우터 (의도 분류 + 4갈래 분기)
- LangGraph 2개 그래프 (Intelligence · SEARCH RAG)
- 임베딩 파이프라인 + pgvector 저장
- RAG 검색 답변 생성 (할루시네이션 4중 방어)
- 유사 영상 추천 (영상 단위 dedup)
- SAVE_ONLY 메타데이터만 저장 로직
- 트러블슈팅 — Celery `.on_error()` 인자 자동 주입 이슈 해결

아키텍처 (한 줄)

카톡 → 웹훅(즉시 ACK) → Redis → Celery 워커 → AI 라우터(4갈래)
→ Intelligence 그래프(요약·임베딩·Overview) → 노선 페이지
또는 SEARCH RAG 그래프(벡터 검색·LLM 답변) → SMTP 메일

핵심 성과 (정량)

지표	값
카카오톡 5초 응답률	100%
영상 1편 자동 요약 시간	약 30~120초 (추정)
검색 응답 시간	약 5~10초 (추정)
SAVE_ONLY 비용 절감	LLM 호출 0회 (UPLOAD 대비 100% 토큰 절감)
할루시네이션 방어	4중 (임계값·프롬프트·출처·빈결과 분기)
트러블슈팅 해결	침묵 실패 1건 (Celery on_error)

🎯 핵심 설계 결정 5가지

1. **AI 라우터를 LLM 구조화 출력 기반으로 설계** — 룰 기반(키워드 매칭) 대신 자연어 이해로 부정문·인용까지 분류
2. **요약본을 임베딩** (원본 자막 대신) — 노이즈 제거로 검색 변별력 확보
3. **거리 임계값 필터링** (RAG 0.7 / 유사 영상 0.8) — 할루시네이션 사전 방어
4. **채널 분리 정책** (영상 요약→노션 / 검색 결과→메일 / 카톡엔 ACK) — 챗봇 발송 비용 0 + 영구 보관
5. **FIND_SIMILAR을 UPLOAD 파이프라인 재사용 + 플래그 통합** — 코드 중복 제거

🔥 핵심 기술 스택 (Top 6)

LangGraph · FastAPI · Celery + Redis · pgvector · OpenAI text-embedding-3-small (1536-d) · Docker Compose

🔧 트러블슈팅 한 줄

3단계 Celery chain의 `.on_error()` 핸들러가 호출되지 않고 사용자에게 아무 안내도 가지 않는 **침묵 실패** 발생 → Celery 워커 로그·공식 문서 정독으로 `.on_error()` 콜백이 `(request, exc, traceback)` 3개 인자를 **자동으로 주입**한다는 사실 발견 → 시그니처 확장으로 해결 → 파이프라인 실패 시 DB FAILED 마킹 + 사용자 안내 메일 정상 동작.

📝 회고 (잘한 결정 / 다시 한다면)

잘한 결정: - LangGraph 도입 → 조건부 분기·노드별 디버깅·LangSmith 트레이싱 동시 확보 - 요약본 임베딩 → 자막 노이즈 제거로 검색 정확도 우위 - 채널 분리 정책 → 챗봇 비용 0 + 결과 영구 보관

다시 한다면: - SEARCH 그래프를 처음부터 async로 → 동시성 향상 - 영상 단💎